CONTENTS
1 Core Concept
2 Data Flow
3 File Genealogy
4 Code Similarity
5 Cross-Dependency
6 Wire-Ready
7 Shared Artifacts
8 Agent Map
9 Pattern Guide
10 Cache Strategy
11 What's Missing
12 File Locations

File Genealogy & Code Similarity

where they live, what they produce, and how Phase 1 agents now see file relationships

1 — CORE CONCEPT

Two powerful systems exist. One now feeds Phase 1.

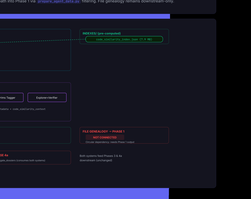
File Genealogy detects when the AI rewrites files under new names — grouping 60 files into 12 concepts. **Code Similarity** compares every Python file against every other, finding 14,176 non-trivial matches across 341 files using AST fingerprints and 6 pattern types.

As of Feb 25, 2026, **code similarity now feeds Phase 1 agents** via a filtered per-session context file. File genealogy remains disconnected due to a circular dependency.

341	57,970	14,176	1 of 2
FILES SCANNED	FILES IGNORED	NON-TRIVIAL MATCHES	ON TOKEN PAGES

2 — DATA FLOW

This diagram shows the updated pipeline. Code similarity now has a path into Phase 1 via `prepare_agent_data.py`. Filing. File genealogy remains downstream-only.



3 — FILE GENEALOGY

What It Does

It doesn't remain files — it rewrites them under new names. A user sees 40 files and feels overwhelmed. File genealogy shows them: "You actually have 12 concepts, and here's the latest version of each." It detects file identity across renames, rewrites, and duplications using three independent signals.

Three Detection Signals

- **Idea-graph lineage** — Concept X (mentions file X) evolved into concept B (mentions file Y) via this graph edge, strongest signal
- **Temporal succession** — File X stops being modified, file Y starts at the same time (within 5 sessions)
- **Name similarity** — Files with overlapping stems. `thread_extractor.py` and `thread_synthesizer.py`: share the core stem.

Implementation

- `phase_2_synthesize/file_genealogy.py` — 400 lines
- Inputs: `thread_extraction.jsonl` (Phase 1 + `idea_graph.jsonl` (Phase 2)
- Output: `file_genealogy.jsonl` (per-session)
- Algorithm: Union-find (mystery) with strict merge rules
- Consumed by: Phase 3 `outlines/file_evidence.py` and Phase 4a `aggregate_decisions.py`

Missing Orchestration

`run_all.sh` (lv) Phase 2 runs `idea_graph.py` and `synthesize.py` — but does **NOT** call `file_genealogy.py`. It must be run manually after Phase 2. If it's not run, Phase 3 gets empty genealogy data.

4 — CODE SIMILARITY

What It Does

Compares all Python files against all others. No pre-filtering. No genealogy input. Completely independent of chat data. Reads source files on disk, extracts AST fingerprints (functions, classes, imports, constants), and classifies every pair into one of 10 relationship patterns.

Five Fingerprint Signals Per File

- **Functions** — AST-normalized top-level functions
- **Class names + method signatures** — AST classes and their methods
- **Import fingerprint** — What modules are imported
- **Constants and string literals** — ALL_CAPS names, strings >10 chars
- **Filename stem words** — Split by underscore

Implementation

- `phase_2_synthesize/code_similarity.py` — 520 lines ● Stable
- `phase_2_synthesize/code_similarity.jsx` — 777 lines ● Duplicate
- Input: Raw Python files on disk (independent of chat pipeline)
- Output: `code_similarity_index.jsonl` (19 MB, 14,176 matches) `/mnt/user-data/outputs/` ⚠
- Consumed by: **Phase 0** `prepare_agent_data.py` (NEW), Phase 4a `aggregate_decisions.py`

The Duplicate Problem

Two near-identical implementations exist in `phase_2_pkg/` and `phase_2_synthesize/`. No comments indicate why or which is canonical. This creates maintenance burden and divergence risk.

5 — CROSS-DEPENDENCY

File Genealogy Cannot Be a Phase 1 Input

File genealogy requires `thread_extraction.jsonl` (a Phase 1 output) to build file activity timelines. It also requires `idea_graph.jsonl` (a Phase 2 output) for its strongest signal: idea-graph lineage. This creates a hard circular dependency.

SIGNAL	REQUIRES	AVAILABLE AT PHASE 0?	NOTES
Idea-graph lineage	`idea_graph.jsonl` (Phase 2)	✗ No	Needs concept evolution edges. Only available after full synthesis.
Temporal succession	File mention timestamps	✗ No	MetadataExtractor already extracts file mentions per message.
Name similarity	Filename list	✓ Yes	All file mentions in Phase 0 metadata. Pure string comparison.

A "file genealogy" running at Phase 0 using only temporal succession and name similarity would give Phase 1 agents partial file relationship data. This **has not been implemented**.

6 — WIRE-READY ARTIFACTS

Phase 0 Changes — `prepare_agent_data.py`

- **New step 11** Loads `code_similarity_index.jsonl` from `INDEXES_DIR`
- Collects all filenames from `session_state.files_seen_ever`
- Filters matches where **both** `file_a` and `file_b` appear in the session's file set
- Also includes `file_hints` entries (fingerprints, isolation_score, shared_match, pattern_summary)
- Writes `code_similarity_context.jsonl` to the session output directory
- Handles missing index gracefully: warning + empty context file (not a hard error)

Phase 1 Changes — `runs_on_phase1.py`

- `load_session_data()` now loads `code_similarity_context.jsonl` alongside the other 3 data files
- All 4 agent prompt functions receive `code_similarity_context` as a keyword parameter
- `_load_code_ctx(ctx)` includes the context in every chunk for large sessions
- All 4 call sites in `phase1_sessions()` pass the context

Agent-Specific Instructions

AGENT	INSTRUCTION FOCUS
Thread Analyst	"Use file relationships to group related threads — e.g., messages about dead copies or evolution pairs likely belong to the same software/code thread."
Geological Reader	"File relationship patterns are geological signals: dead_copy = fossilized layer, evolution_pair = sedimentation drift, function_clone = mineral vein across strata."
Primitives Tagger	"Use file relationship patterns to inform tagging — dead copies may indicate cleanup intent, evolution pairs suggest refactoring action, vector."
Explorer + Verifier	"Check whether the other agents correctly identified dead copies and evolution pairs. Flag any dead_copy files the Thread Analyst treated as independent work."

- `Output Schema` → `code_similarity_context.jsonl`

7 — SHARED ARTIFACTS

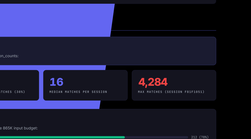
Coverage Across 272 Sessions

Simulated filtering the global index against every session's `file_mention_counts`:

190	82	16	4,284
SESSIONS WITH MATCHES (70%)	SESSIONS WITH 5 MATCHES (30%)	MEDIAN MATCHES PER SESSION	MAX MATCHES (SESSION f8d7e51)

Token Budget Distribution

Estimated token cost of `code_similarity_context` as a percentage of the 8K token input budget:



The Budget Bomb Problem

3 sessions (mentioning 375-816 files) would consume **50-87% of the 8K token budget** with code similarity context alone, leaving almost no room for actual message data. Current implementation has **no cap**.

Worst case: session_f8f9201 (293 files, 4,284 matches) → ~7K tokens = 87% of budget.

Design decision needed — options:

- **Option 1: Signal score threshold** — Only include matches above score X (e.g., >10)
- **Option 2: Match count cap** — Max N matches per session (e.g., 100)
- **Option 3: Token budget cap** — Limit context to X% of total budget (e.g., 10%)
- **Option 4: Truncate shared_functions** — Remove function name lists for large contexts

The Dead Copy Paradox

The global index contains 16 **dead copies** (all `gdt_enforcer.py` vs `test_gdt_enforcer.py` pairs). But **0 dead copies survive session filtering** — users interact with one copy, never both in the same session. The most actionable pattern for Phase 1 is actually **function_clone** (7,069 matches across sessions).

What Agents Gained — Test Session 00f2e0ed

Compared existing Phase 1 output (generated without code similarity) against what the context would have shown:

- **Thread Analyst missed** a triangle of 3 files (`devlain_trainer.py`, `deep_relay_trainer.py`, `bolt_master.py`) that share function clones with score 14. Code similarity context would have revealed this.
- **Explorer found only 1 of 12 observations** related to file relationships. Context would have enriched verification significantly.
- **8 high-signal matches** (score >5) the agents were completely blind to.
- **237 total matches** for 54 session files, consuming ~48K tokens (5.1% of budget) — well within acceptable range.

8 — AGENT MAP

BEFORE (CURRENT)	AFTER (CURRENT)
• Message content (full text, all lens) • Metadata (files mentioned, errors, cwds, cats) • Opus-assigned lens (critical/significant/context/noise) • Behavior flags (constant change, confusion) • Protocol detection (ia_protocol, protocol_type) • Session metadata (token counts, message counts) • **No file relationships. No duplicate detection.**	• Everything from before, plus: • `Code similarity matches in session mentioned files` • `Pattern types per pair (10 types)` • `Key fingerprints (functions, classes, imports)` • `Isolation scores per file` • `Shared match per match` • **No token budget cap yet. No file genealogy.**

9 — PATTERN GUIDE

PATTERN	TYPEDNESS	WHAT IT MEANS	ACTION
`dead_copy`	>90% text	Virtually identical file. One is redundant.	● Safely one
`evolution_pair`	80-90% text	Same file diverged over time. Version drift.	● Review/Pick
`function_clone`	>50% func, <90% text	Copy-paste with edits. Shared function names.	● Extract shared
`partial_overlap`	>80% containment	One file's functions are a subset of the other's.	● Fix design
`shared_origin`	>40% callable, <30% text	Same structure, different content.	● Fix design
`import_hub`	>70% imports, <30% func	Same ecosystem, different purpose.	● Intentional
`name_only`	>30% name, <5% code	Misleading names. Filenames overlap but code doesn't.	● Intentional
`interface_similar`	3+ shared func, <30% text	API drift. Same function names, divergent implementations.	● Fix drift
`structural_sibling`	>5% callable, >30% imports	Same general structure and ecosystem.	● Intentional
`signal_ecosystem`	>40% imports, >10% callable	Part of the same import ecosystem.	● Intentional

Scoring Formula

```
signal = name_score + 3 * callable_overlap + 2 * class_overlap + 1 * import_overlap + 1 * constant_overlap + 1 * exec_overlap + 1
```

Text comparison (difflib) only runs on pairs where `signal_score ≥ 3.5` or `name_score ≥ 0.3`. This optimization means ~80% of the 57,970 pairs avoid the expensive text diff step.

10 — DATA SCHEMAS

`file_genealogy.jsonl` — Per-Session Schema

```
session_id string — Session UUID
file_families[] array — Grouped files representing the same concept
  .canonical — Human-readable concept name
  .versions[] — Each version: file, status (current/superseded), active_msgs, events, evidence
  .latest — Name of current version
  .standalone_files[] — Files with no detected relatives
  .lens_breakdown — Counts by signal type: idea_graph, temporal, name_similarity, total
  .newcomer — E.g. "60 files → 45 concepts"
```

`code_similarity_index.jsonl` — Global Index Schema

```
total_files  integer (341) | total_pairs_compared  integer (57,970) | non_trivial_matches  integer (14,176)
pattern_distribution  — Counts per pattern type
matches[]  — Every non-trivial pair: file_a, file_b, signals (fn fields), patterns[]
file_stats{}  — Per file: fingerprint, total_matches, pattern_summary, strongest_match, isolation_score
```

`code_similarity_context.jsonl` — Per-Session Filtered Context

```
session_id  string — Session UUID
generated_at  ISO timestamp — When context was generated
source_index  — "code_similarity_index.jsonl"
session_files_count  integer — Files mentioned in session
matches_included  integer — Number of filtered matches
matches[]  — Filtered matches where both file_a and file_b are in session file set
  .file_a, .file_b  — Filenames
  .signals  — 11 fields: fn_overlap, callable_overlap, class_overlap, import_overlap, constant_overlap, name_overlap, text_similarity, containment_a_in_b, containment_b_in_a, shared_functions[], signal_score
  .patterns[]  — 0-4 of the 10 pattern types
  .file_stats{}  — Per session-mentioned file: fingerprint, total_matches, pattern_summary, strongest_match, isolation_score
```

11 — WHAT'S STILL MISSING

1. No Token Budget Cap
10 sessions (7.4%) fall consuming >20% of the token budget with code similarity context. 3 sessions would consume >50%. **Design decision needed.**

2. File Genealogy Still Disconnected
Circular dependency unresolved. The "file genealogy" idea (temporal succession + name similarity) at Phase 0) remains unimplemented.

3. Duplicate code_similarity.py
`phase_2_pkg/code_similarity.py` (520 lines) and `phase_2_synthesize/code_similarity.jsx` (777 lines) — near-identical. Which is canonical? No resolution yet.

4. file_genealogy.py Not in run_pipeline.py
Must be run manually after Phase 2. If not run, Phase 3 gets empty genealogy data and lineage sections are blank.

5. No Dead Copy Visibility for Phase 1
Session filtering requires **both** files to be mentioned. Since dead copies are never co-mentioned, Phase 1 agents will never see dead_copy alerts. Consider including dead copies where **either** file is mentioned.

6. No Signal Score Threshold
Low-signal matches (score <5 add noise without actionable insight. A minimum threshold would reduce token cost and improve signal-to-noise ratio.

12 — FILE LOCATIONS

FILE	LINES	PHASE	PURPOSE
`phase_2_synthesize/file_genealogy.py`	400	Phase 2	File genealogy detector (primary)
`phase_2_pkg/code_similarity.py`	520	Phase 2	Code similarity full scan (primary)
`phase_2_synthesize/code_similarity.py`	777	Phase 2	Code similarity full scan (duplicate)
`partial/code_similarity_incremental.py`	272	Standalone	Incremental code similarity with caching
`phase_0_prep/prepare_agent_data.py`	370	Phase 0	Now generates code_similarity_context.jsonl
`phase_1_extract/runs_on_phase1.py`	905	Phase 1	Now loads and injects context into all 4 agents
`phase_3_human/apply_agent_decisions.py`	800+	Phase 3	Consumes file_genealogy.jsonl
`phase_4a_aggregation/aggregate_decisions.py`	760+	Phase 4a	Consumes both genealogy and similarity
`update_schemas.py`	130	Tools	Companion script for pipeline schematic updates